



ARC Training Centre for Behavioural
Insights for Technology Adoption

ARC BITA Geek Seminars

Git + Python (Poetry):
Hands-on Foundations

18 September 2025



Dr. Steve Bickley

s.bickley@qut.edu.au

<https://arcbita.org/>



Acknowledgement to Country

I acknowledge the **Turrbal** and **Yugara** as the First Nations owners of the lands where we now stand (*Meanjin*).

I pay respect to their Elders, lores, customs and creation spirits, and recognise that these lands have always been places of teaching, research and learning.

I acknowledge the important role Aboriginal and Torres Strait Islander people play within the QUT and wider community.

Welcome & Context

Welcome and thank you for your attendance today!

Slides Available at: <https://github.com/stevejbickley/arcbita-geek-seminars>

Why Python?

- Readable, beginner-friendly syntax → faster onboarding & fewer bugs
- Cross-platform (macOS/Windows/Linux) with strong IDE/editor support
- Large ecosystem (PyPI) for data, AI/ML, web, automation, and research
 - Also note, LLMs / ALMs also heavily geared towards Python code generation
- “Batteries included” standard library for everyday tasks
- *Tooling*: Poetry (deps/lockfiles), pytest (tests), Ruff/Black (style), ...
- *Collaboration*: clear packaging, scripts, and CI/CD integration (continuous integration & continuous delivery/deployment)
- **Python toolchain with pyenv + Poetry**
 - pyenv
 - Manages multiple Python versions per project/user.
 - Poetry
 - Dependency resolver, virtualenvs, scripts, build/publish.
 - Why together?
 - Pin Python version + lock dependencies for reproducibility.

Why Git?

- Version history & safety
 - Roll back mistakes, experiment safely
- Branching & collaboration
 - Parallel work without conflicts (in theory!)
- PRs & review culture
 - Discuss changes, document decisions
- Portability & ecosystem
 - Works with GitHub/GitLab/Bitbucket → CI/CD pipelines
- Reproducibility
 - Lock project state alongside code

Core Python concepts

- **Interpreter & REPL** (Read-Eval-Print Loop – run one line at a time) **vs running scripts** (python file.py)
- **Modules, packages & imports w/ entry points** via e.g. pyproject.toml scripts
- **Virtual environments & dependency locking** (e.g. Poetry + poetry.lock)
- **Types & collections:** int/float/str/bool/None, list/tuple/dict/set
- **Control flow & iteration:** if/elif/else, for/while, comprehensions
- **Functions & scope** (local/enclosing/global/built-in) + type hints (def f(x: int) -> str)
- **Classes & dataclasses** for lightweight models + dunder or double underscore methods (e.g., `__init__`, `__str__`, `__len__`, `__getitem__`)
 - **Key difference:** data classes are for lightweight “records/containers,” while regular classes are more general and flexible (for logic, behaviour, inheritance, etc.)
- **Exceptions & testing:** try/except, pytest

```
squares = []
for x in range(5):
    squares.append(x**2)

#
squares = [x**2 for x in range(5)]
```

`__init__`: runs when you create an object.
`__str__`: defines how an object prints.
`__len__`: lets you use `len(obj)`.
`__getitem__`: makes objects indexable like lists.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Core Git concepts

- **Repository**

- Local (lives on your machine) vs remote (hosted on e.g. GitHub)
- Workflow: you commit locally, then git push to share your changes with the remote repo. To bring in others' changes, you git pull (fetch + merge)

- **Commit**

- A snapshot of your project at a point in time, saved w/ a message
- *Best practice*: make commits small, logical, and purposeful (e.g. "fix bug in login validation" instead of "misc changes")

- **Branch**

- A **branch** is a movable pointer to a commit, used to isolate work
- E.g., main/ or master/ (production-ready), dev/ (integration/testing), feature/* (one branch per new feature)

- **Remote / forked workflows**

- **Origin**: By convention, the name of your *own remote copy* of a repo (the one you cloned from or your personal fork)
- **Upstream**: The *main project's repository*, usually the original repo you forked from. You add it separately so you can keep your fork in sync
- Typical flow in forked workflows:
 - Pull changes from upstream → merge into your local main branch
 - Push that updated main branch to your origin
 - Create or switch to a feature branch → push it to origin
 - Open a Pull Request from your feature branch into the upstream project.

- **Pull Request (PR)**

- A PR is a request to merge your branch into another (often main)
- Allows discussion, review, and automated checks before changes go in
- Think of it as the *collaborative gate* where others can review your work

Typical repo workflow

- Clone
 - `git clone <starter-repo-url>`
- Create branch
 - `git checkout -b feature/hello-poetry`
- Make change
 - Edit files, run tests.
- Commit & push
 - `git add -A` or `git add .`
 - `git commit -m "feat: initial demo"`
 - `git push -u origin feature/hello-poetry`
- Open PR
 - Review → merge

Hands-on Foundations *(Installs)*

Feel free to just observe/follow along and watch the recording later

Slides Available at: <https://github.com/stevejbickley/arcbita-geek-seminars>

First, always check your toolchain!

```
# Versions (any platform e.g., Pycharm, terminal/command line, etc.)
git --version
python --version
poetry --version    # after install
pyenv --version     # after install
```

Don't worry.. we will install together if missing!

Install Git, pyenv & Poetry

Git

- **macOS:** brew install git (or Xcode Command Line Tools prompt)
- **Windows:** Download installer from git-scm.com, enable “Git from the command line” during setup
- **Linux:** sudo apt-get install git (Debian/Ubuntu) or sudo yum install git (Fedora/RHEL)

pyenv

- **macOS (Homebrew):**
 - brew update
 - brew install pyenv
 - echo 'eval "\$(pyenv init -)"' >> ~/.zshrc
- **Windows (pyenv-win):** Install pyenv-win; ensure PATH updated; restart terminal
- **Linux:** Use your distro’s package manager or official installer

Poetry

- **macOS/Linux:** curl -sSL https://install.python-poetry.org | python3 -
- **Windows (PowerShell):** (Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | py -
- **Recommended:** keep virtualenvs in-project → poetry config virtualenvs.in-project true

pyenv handles Python version management/installs during the session:

pyenv install <version>

• On Linux/macOS, pyenv compiles Python versions from source. That means you need system build tools:

• **macOS:** Xcode Command Line Tools (xcode-select --install).

• **Ubuntu/Debian:** sudo apt-get install build-essential libssl-dev zlib1g-dev libbz2-dev libreadline-dev libsqlite3-dev curl (etc).

• **Fedora/RHEL:** similar dnf/yum development libraries.

• Without these, pyenv install <version> often fails with cryptic errors.

Version naming

- Use the exact version string available (pyenv install --list shows all). Sometimes 3.11.9 vs 3.11.10 matters.

Troubleshooting: Common Errors & Quick Hits

- **PATH not updated?**
 - Close/reopen terminal; check echo \$PATH / \$Env:Path
- **Multiple Pythons?**
 - pyenv which python; poetry env info
- **Poetry not found?**
 - Check installer output; add to PATH; restart shell
- **SSL/proxy issues?**
 - Use system proxy vars; consider alt installer
- **Windows: Git Bash vs PowerShell confusion**
 - Reminder to run commands in the same shell you set up with (PATH differences can bite)
- **Permission errors on Windows (admin rights)**
 - Especially for QUT laptops, flag that *makemeadmin* might need to be active when installing/updating
- **zsh vs bash init files on macOS/Linux**
 - Sometimes you may edit .bashrc when your shell is actually zsh, or vice versa. Worth a one-liner reminder
- **Proxy/firewall issues on institution networks**
 - Curl/PowerShell installers can fail if behind a proxy.. some users may need to retry on home Wi-Fi

Hands-on Foundations *(Basics)*

Feel free to just observe/follow along and watch the recording later

Slides Available at: <https://github.com/stevejbickley/arcbita-geek-seminars>

Git: first-time config (run once)

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"  
git config --global init.defaultBranch main
```

```
# Line endings (recommended)  
git config --global core.autocrlf input    # macOS/Linux  
git config --global core.autocrlf true     # Windows
```

Git Clone Methods (HTTPS vs SSH)

HTTPS (default, easier):

```
git clone https://github.com/user/repo.git
```

SSH (requires SSH key setup):

1. Generate SSH key: `ssh-keygen -t ed25519 -C "your_email@example.com"`
2. Add key to ssh-agent & copy to GitHub profile (Settings → SSH and GPG keys)
3. Ensure key path added to your PATH/agent
4. Clone with: `git clone git@github.com:user/repo.git`

GitHub CLI (optional):

```
gh repo clone user/repo
```


Typical Git Workflow & .gitignore hygiene

Typical workflow

- Clone
 - `git clone <starter-repo-url>`
- Create branch
 - `git checkout -b feature/hello-poetry`
- Make change
 - Edit files, run tests.
- Commit & push
 - `git add -A` or `git add .`
 - `git commit -m "feat: initial demo"`
 - `git push -u origin feature/hello-poetry`
- Open PR
 - Review → merge

Add .gitignore file to your repo

```
# Python
__pycache__/
*.py[cod]
*.egg-info/

# Poetry virtualenv in project
.venv/

# OS noise
.DS_Store
Thumbs.db
```

Merge vs rebase

- Merge
 - Keeps full history; creates merge commit
- Rebase
 - Linear history; rewrites commits onto target branch
- Rule of thumb
 - Prefer merge for teams
 - rebase local feature branches before PR if repo policy allows

Resolve simple conflicts (via PR)

- Pull latest main → update your branch
- Fix conflicts in editor
 - (markers <<<<<<< =====>>>>>>>)
- Re-test, commit, push; request review

Create project with Poetry

```
poetry new hello-poetry
cd hello-poetry
pyenv install 3.11.9      # example version
pyenv local 3.11.9
poetry env use python     # bind Poetry to local Python
poetry add requests       # add a dependency
```

```
# Add a script (pyproject.toml)
[tool.poetry.scripts]
hello = "hello_poetry.__main__:main"
```

Then we can run & test:

```
poetry run python -m hello_poetry
poetry run pytest          # if tests added
poetry add pytest --group dev
poetry lock                # ensure lockfile present
```

Wrap up & Next steps

Thank you again for your attendance today!

Slides Available at: <https://github.com/stevejbickley/arcbita-geek-seminars>

What we achieved / Hands-on lab checklist

- Clone starter repo and create feature branch
- Create or edit a small script/test
- Commit and push + open a PR
- Resolve a simple conflict and merge
- Poetry: add a dependency, run script with poetry run

Next steps

- Keep the template repo as your starting point
- Open issues/PRs against the starter for questions
- Try adding linting/formatting (ruff/black) and CI/CD (Continuous Integration & Continuous Delivery/Deployment) **next..**